
Technical Notes for heliotis gen4 Devices

Release 2025.09

Heliotis AG

Aug 29, 2025

OVERVIEW

1	TN_15.1.0.1: heliBrd B4 SD Card Re-Writing	3
1.1	Challenge: Corrupt SD Card Content	3
1.2	Solution: SD-Card Re-Writing	3
1.2.1	Remove SD-Card from heliInspect H8 or heliCam C4	3
1.2.2	Load an SD-Card Image	5
1.2.3	Insert an SD-Card to heliInspect/heliCam Device	6
1.3	Document History	7
2	TN_15.2.0.0: Adjust 3D Data Scaling and Offset	9
2.1	Introduction	9
2.1.1	Challenge	9
2.2	Configure Custom Surface Scaling	9
2.3	Configure Custom Surface Offset	10
2.4	Calculate Real-World Coordinates Data	10
2.5	Document History	14
3	TN_15.3.0.1: Collecting and Interpreting Monitoring Information of gen4 Devices	15
3.1	Context and Challenge: Finding Root Causes of Non-Systematic Issues	15
3.2	Solution: Client-Based System Monitoring	15
3.2.1	Permanently Available Status Information	16
3.2.2	Status Information Relevant after Configuration	16
3.2.3	Status Information of the Acquisition Engine	17
3.2.4	Status Information Related to Measurement Data	17
3.3	Document History	18
4	TN_19.1.0.1: UserSet - A short Introduction	19
4.1	Context and Challenge: Storing gen4 Device Configurations	19
4.2	Solution: UserSet	19
4.2.1	Storing (or Creating) a UserSet	19
4.2.2	Loading a UserSet	20
4.2.3	Default UserSet	20
4.2.4	File Transfer	20
4.2.5	Additional Features	21
4.3	Document History	22
5	TN_19.10.0.1: Configuring Offset (FPN) Correction	23
5.1	Challenge: FPN Removal as a Necessary Pre-Processing Step	23
5.2	Solution: Configure a Frame Range for Averaging Sufficiently Far from the Surface	24
5.3	Implementation	25
5.4	Document History	25
6	TN_19.2.0.1: Configuring the Optic Type	27
6.1	Context: Variety of Optics Types for heliInspect H8/H9	27
6.2	Challenge: Correctly Configure heliInspect H8/H9 for an Optics Type	27
6.3	Solution: Load Predefined Configurations using the OpticType Feature	27

6.4	Document History	28
7	TN_19.3.0.1: Reducing Vibration-Induced WLI Amplitude Loss	29
7.1	Context: Ideal Scanning of Sample-to-Optic Distance	29
7.2	Challenge: Amplitude Drop due to Velocity Deviations	29
7.3	Fundamentals: Correct Wavelength Setting and Avoidance of Velocity Variations	30
7.4	Parametric Solution: Increase ScanSpeed and Decrease VerticalSpacing for Improved Robustness	30
7.5	Extended Parametric Solution: Low VerticalSpreading Enables Very Short Observation Interval	31
7.6	Practical Implementation	32
7.7	Document History	33
8	TN_19.4.0.1: Scan Motion Monitoring (zTag-reading)	35
8.1	Challenge: Monitoring Scan Motion Profile	35
8.2	Implementation	36
8.3	Interpretation	37
8.4	Document History	38
9	TN_19.9.0.0: First/Last Surface Extraction	39
9.1	Challenge: Interference Envelope of Transparent Samples	39
9.2	Solution: Amplitude Peak Detection and Thresholding	39
9.3	Tips for Narrow Transparent Layers	40
9.4	Implementation	40
9.5	Document History	40
10	TN_51.1.0.0: Heliotis Gen4 Device Fieldbus Reference for D3	41
10.1	Introduction	42
10.2	Example Use Cases	43
10.2.1	“Hello World”	43
10.2.2	Camera Status	44
10.2.3	Acquisition Control	45
10.2.4	Camera Control, Light ON/OFF	45
10.2.5	Camera Control, Sequencer Sets	46
10.2.6	Camera Control, Counters	46
10.2.7	Vision Application, Parameterisation	47
10.2.8	Vision Application, Result Transmission	48
10.3	Further Reading	49
10.3.1	Heliotis	49
10.3.2	Beckhoff	49
10.3.3	Siemens	49
10.4	Document History	50

This is a collection of technical notes for the heliotis gen4 product generation.

TN_15.1.0.1: HELIBRD B4 SD CARD RE-WRITING

1.1 Challenge: Corrupt SD Card Content

heliBrd B4 based devices (like heliInspect H8, heliCam C4) store a file system on an SD Card. Updates of this file system are handled using the UpdateTool application (comes with C4Utility installer) and image package or update package files (available for download on heliotis webpage). The file system is robust, i.e. it tolerates conditions such as power interruption during use. Only in exceptional failure cases the file system may be corrupted to the point of boot failures. In such rare cases, however, the device becomes inaccessible through its ethernet interface even after attempts to reboot it (power cycle).

Note: If you suspect your file system to be corrupt contact support@heliotis.ch before proceeding with solution described below.

1.2 Solution: SD-Card Re-Writing

To recover from a corrupt file system load a full SD-Card image to the device. The required steps are as follows.

1.2.1 Remove SD-Card from heliInspect H8 or heliCam C4

Required tools (figure Fig. 1.1):

- 1.5mm hexagonal socket wrench (Allen key)
- tweezers

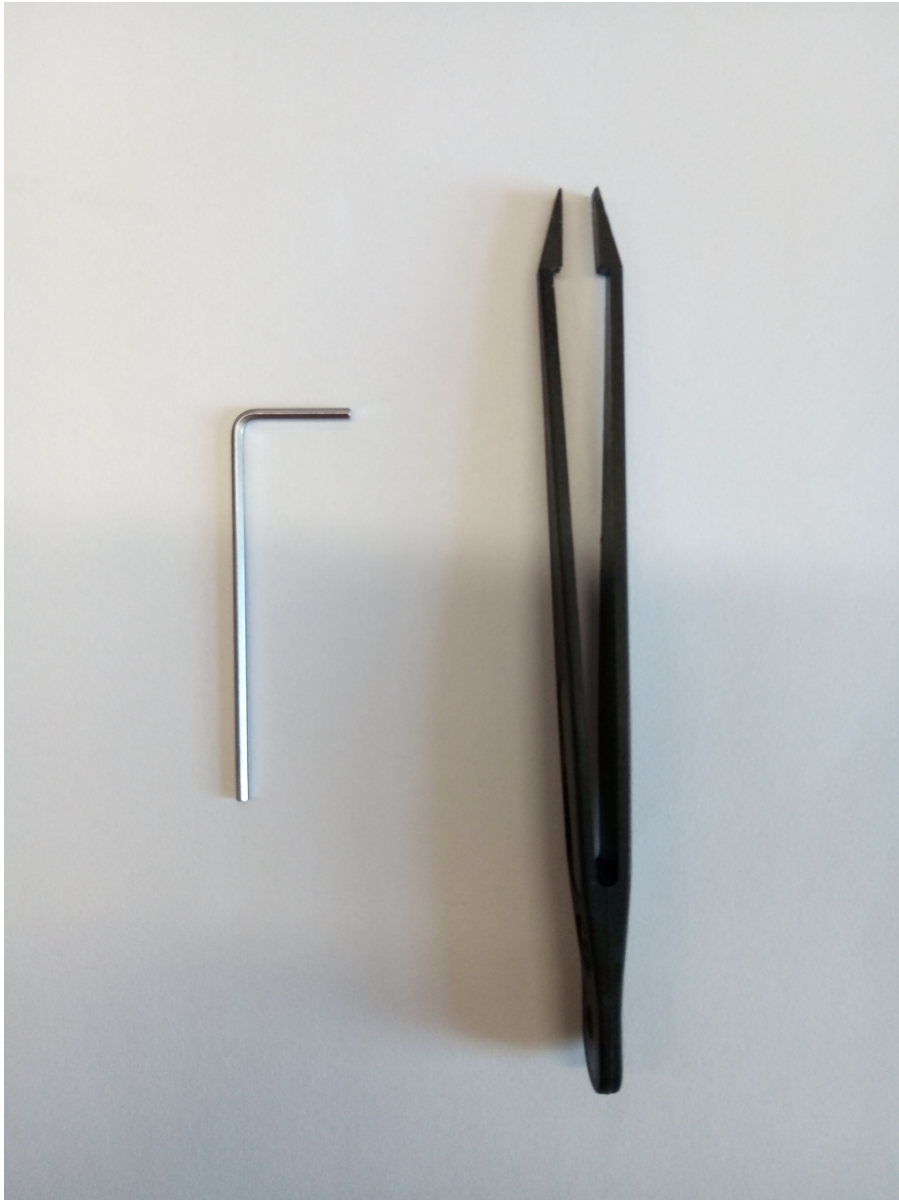


Fig. 1.1: Required Tools

Steps:

1. Switch off the power supply.
2. Remove SD-Card cover. Unscrew the two screws on the right hand side of heliInspect H8 (figure [Fig. 1.2](#)). Remove the cover.
3. Remove the SD-Card. The SD-Card holder has a push/pull mechanism. To remove the SD-Card, push the card with a thin rod (e.g. the back of your tweezers). A soft click should be audible. Afterwards, the card pops out a bit and can be removed carefully. (e.g. with the tweezers)



Fig. 1.2: SD-Card Cover Screws

Note: SD card is not accessible for heliInspect H9 devices. Don't attempt to open H9 devices without heliotis assistance.

1.2.2 Load an SD-Card Image

Required software:

- balenaEtcher <https://www.balena.io/etcher/>
- heliBoard B4 Image (e.g. b4brdImg_1.8.0.zip)

Steps:

1. Extract the *.zip file. (It contains a single *.img file)
2. Insert the card into PC's SD-Card slot and start balenaEtcher

3. Push the *Flash from file* button and select the extracted *.img file
4. Push the *Select target* button and select the SD-Card drive.
5. The final selections are shown in figure Fig. 1.3. Now push the *Flash!* button to write the image to the SD-Card (PC administrator privileges are required)
6. After a successful write procedure, remove the SD-Card from the PC and insert it into the heliInspect H8.

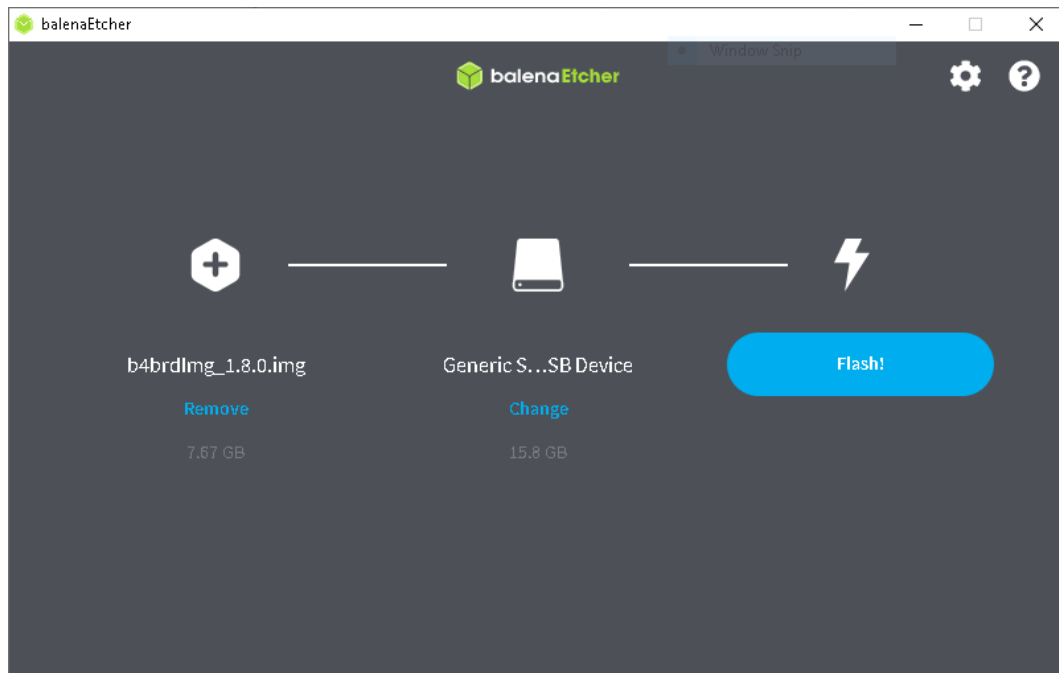


Fig. 1.3: balenaEtcher selection

1.2.3 Insert an SD-Card to heliInspect/heliCam Device

Steps:

1. Insert the SD-Card into heliInspect H8. Use tweezers to hold the card and insert it carefully into the SD-Card slot. The SD-Card slot has a push/pull mechanism. After inserting the SD-Card push it carefully until a soft click is audible. The SD-Card orientation is shown in figure Fig. 1.4.
2. Mount the cover with the two screws.



Fig. 1.4: SD-Card Orientation

1.3 Document History

Version	Date	Changes	Responsible
15.1.0.0	19-05-2022	initial version	sm
15.1.0.1	14-11-2022	review, typos	cl

TN_15.2.0.0: ADJUST 3D DATA SCALING AND OFFSET

2.1 Introduction

heliInspect devices scale 3D output data in *um* and map the value to the current stage position which is in many cases a negative position

This application note describes how to setup 3D data scaling and offset. This allows a well configured value range for *unsigned int16* (Mono16) surface output.

The required features described in this documentation are available since software version **1.9.0**.

2.1.1 Challenge

Some applications could not handle the 3D data output as 32bit floating point value. For such applications, the heliInspect H8/H9 as well as any other heliBrd B4 based camera systems from heliotis supports an 16bit unsigned integer data output. Within this mode, the output values are integers and its range is limited from 0 - 65535.

As a default configuration, the camera scales the 3D output data in *um* and maps the value to the current stage position which is in many cases a negative position. In section [Configure Custom Surface Scaling](#) the configuration for custom output data scaling is described. Section [Configure Custom Surface Offset](#) contains the configuration for a custom surface shift along the z direction and finally, the section [Calculate Real-World Coordinates Data](#) contains the calculations and additional information how the submitted 3D data are transferred back to the *real-world* data.

Note: The Scaling and Offset configuration is applied to any 3D Data output data (16bit integer as well as 32bit floating point).

2.2 Configure Custom Surface Scaling

A rough scaling setting is applied with the feature *Scan3dDistanceUnit* which allows to select the data output unit as *mm*, *um* or *nm*. The default value, and in almost all cases the best selection is *um*.

To adjust a custom scaling the feature *Scan3dCoordinateScale[Scan3dCoordinateSelector]* is available. The reciprocal value of the *CoordinateC* configuration is internally multiplied with the real-world z value. As an example, the following code listing shows the configuration to adjust the z values to 100nm per integer step.

Listing 2.1: Scan3dCoordinateScale example

```
Scan3dDistanceUnit = "um"
Scan3dCoordinateSelector = "CoordinateC"
Scan3dCoordinateScale = 0.1
```

2.3 Configure Custom Surface Offset

The feature *Scan3dCoordinateOffset[Scan3dCoordinateSelector]* allows the shifting of the 3D output data along the z axis. The value configured within the *CoordinateC* selector is internally subtracted from the real-world z values. As an example, the following code listening shows the configuration to shift the the surface 14.1mm upper. This place a real-world pixelvalue from -13900um up to 200um.

Listing 2.2: Scan3dCoordinateOffset example

```
Scan3dDistanceUnit = "um"
Scan3dCoordinateSelector = "CoordinateC"
Scan3dCoordinateOffset = -14100.0
```

Note: The unit of the *Scan3dCoordinateOffset* feature is configured with the *Scan3dDistanceUnit* feature. It is recommended to select a well *Scan3dDistanceUnit* before tuning the *Scan3dCoordinateOffset* value.

2.4 Calculate Real-World Coordinates Data

The calculation of the 3d data to real-world data requires the configuration value of the *Scan3dCoordinateScale[Scan3dCoordinateSelector]* and *Scan3dCoordinateOffset[Scan3dCoordinateSelector]* features. The values are readable from the device configuration by reading the two features directly or optionally included in the data buffers meta data as so-called chunk data.

The first listening [Listing 2.3](#) shows how to read the information from the device configuration and the second listening [Listing 2.4](#) how to enable the chunk data and read the information from the buffer chunk data. For the final calculation one of this two approaches could be selected.

Listing 2.3: read Scan3dCoordinateScale and Scan3dCoordinateOffset example

```
Scan3dCoordinateSelector = "CoordinateC"
scaleC = Scan3dCoordinateScale
offsetC = Scan3dCoordinateOffset
```

Listing 2.4: enable Chunk data and read Scan3dCoordinateScale and Scan3dCoordinateOffset from the data buffer

```
# enable the chunk data
ChunkModeActive = True
ChunkSelector = "Scan3dCoordinateScale"
ChunkEnable = True
ChunkSelector = "Scan3dCoordinateOffset"
ChunkEnable = True

# after an acquisition, the configuration is readable from the buffers chunk data
ChunkScan3dCoordinateSelector = "CoordinateC"
scaleC = ChunkScan3dCoordinateScale
offsetC = ChunkScan3dCoordinateOffset
```

The following pseudo code shows the transformation of the submitted 3d data to real-world coordinate.

Listing 2.5: pseudo code how to transform the 3d Data to real-world coordinate data

```
for(row = 0; row < Height; row++)
{
```

(continues on next page)

(continued from previous page)

```

for(col = 0; col < Width; col++)
{
    coordC[row,col] = imageC[row,col] * scaleC + offsetC;
}
}

```

Listing 2.6: test 42

Listing 2.7: Python example demonstrate this application node

```

1  ###
2  # \file          an_15.2.x.y.py
3  # \author       Silvan Murer
4  # \date         04 Aug 2022
5  # \copyright    Heliotis, 2022
6  #
7  # \brief        Example for an_15.2.x.y
8  import os, sys
9  import time
10 from harvesters.core import Harvester
11 import genicam.genapi
12 import matplotlib.pyplot as plt
13 from mpl_toolkits.axes_grid1 import make_axes_locatable
14 import numpy as np
15 import statistics
16 from collections import OrderedDict, Mapping
17 import json
18
19 import warnings
20 warnings.simplefilter('always') # always, ignore
21
22
23 ###
24 # Scan for devices and let the user select one
25 #
26 # \param    h harvesters object
27 # \return   harvesters camera object
28 def selectDevice(h):
29     h.update()
30     print(str(len(h.device_info_list)) + " devices detected on the Network\n\n")
31     cnt = 0
32     for dev in h.device_info_list:
33         cnt = cnt + 1
34         msg = str(cnt) + ") " + dev.id_ + " (sn:" + dev.serial_number + ")"
35         print(msg)
36
37     selection_str = input("Select a device (0=exit): ")
38     selection_int = int(selection_str)
39     if((selection_int <= 0) or (selection_int > cnt)):
40         exit('No device selected - exit script')
41
42     deviceID = h.device_info_list[selection_int-1].id_
43     print('selected device:', deviceID)
44     return h.create(selection_int-1)
45
46
47 ###
48 # initialize the camera with some default parameters

```

(continues on next page)

(continued from previous page)

```

49  #
50  # \param camera camera handler
51  def initializeDevice(camera):
52      # enable required components
53      camera.remote_device.node_map.ComponentSelector.value = "Intensity"
54      camera.remote_device.node_map.ComponentEnable.value = False
55      camera.remote_device.node_map.ComponentSelector.value = "Range"
56      camera.remote_device.node_map.ComponentEnable.value = True
57      camera.remote_device.node_map.ComponentSelector.value = "Reflectance"
58      camera.remote_device.node_map.ComponentEnable.value = False
59      camera.remote_device.node_map.ComponentSelector.value = "Phase"
60      camera.remote_device.node_map.ComponentEnable.value = False
61
62      # trigger configuration
63      camera.remote_device.node_map.TriggerSelector.value = "RecordingStart"
64      camera.remote_device.node_map.TriggerMode.value = "On"
65      camera.remote_device.node_map.TriggerSource.value = "Stage"
66      camera.remote_device.node_map.TriggerSelector.value = "AcquisitionStart"
67      camera.remote_device.node_map.TriggerMode.value = "Off"
68      camera.remote_device.node_map.TriggerSelector.value = "FrameStart"
69      camera.remote_device.node_map.TriggerMode.value = "On"
70      camera.remote_device.node_map.TriggerSource.value = "Software"
71
72      # motion and position configuration
73      camera.remote_device.node_map.ScanPosition.value = -14.5
74      camera.remote_device.node_map.ScanRange.value = 0.5
75      camera.remote_device.node_map.ScanSpeed.value = 5.0
76      camera.remote_device.node_map.GeneralSpeed.value = 10.0
77
78      camera.remote_device.node_map.ScanMode.value = "Down"
79      camera.remote_device.node_map.StageInit.execute()
80
81      # additional camera and processing configuration
82      camera.remote_device.node_map.Scan3dExtractionMethod.value =
83      ↪ "AcceleratedCenterOfMassIQCorrection"
84      camera.remote_device.node_map.Scan3dScalingMethod.value = "zTags"
85      camera.remote_device.node_map.Scan3dDistanceUnit.value = "um"
86
87      camera.remote_device.node_map.TargetVerticalSpacing.value = 2.5
88      camera.remote_device.node_map.ExposureRatio.value = 1.0
89
90      camera.remote_device.node_map.FPNCorrection.value = "AverageLastFrames"
91      camera.remote_device.node_map.FPNCorrectionNFrames.value = 8
92      camera.remote_device.node_map.ExtSimpMaxHWin.value = 7
93
94      heliDriverFamily = camera.remote_device.node_map.HeliDriverFamily.value
95      # illumination control (D3)
96      if (heliDriverFamily == "D3"):
97          camera.remote_device.node_map.LightControllerSelector.value =
98          ↪ "LightController0"
99          camera.remote_device.node_map.LightControllerSource.value = "UserOutput0"
100         camera.remote_device.node_map.LightBrightness.value = 100.0
101         # illumination control (D2)
102         elif (heliDriverFamily == "D2"):
103             camera.remote_device.node_map.LineSelector.value = "Line2"
104             camera.remote_device.node_map.LineSource.value = "UserOutput0"
105             camera.remote_device.node_map.LineInverter.value = 1
106         # switch on the illumination (D2/D3)
107         camera.remote_device.node_map.UserOutputSelector.value = "UserOutput0"
108         camera.remote_device.node_map.UserOutputValue.value = True

```

(continues on next page)

(continued from previous page)

```

108 print('*****')
109 print('* ' + str(os.path.basename(__file__)))
110 print('*****')
111
112 try:
113     ctiFile = os.environ['DIAPHUS_GENTL64_FILE']
114
115     h = Harvester()
116     h.add_file(ctiFile)
117
118     camera = selectDevice(h)
119     initializeDevice(camera)
120     print("INIT DONE")
121
122     # configure application node specific features
123     camera.remote_device.node_map.ChunkModeActive.value = True
124     camera.remote_device.node_map.ChunkSelector.value = "Scan3dCoordinateScale"
125     camera.remote_device.node_map.ChunkEnable.value = True
126     camera.remote_device.node_map.ChunkSelector.value = "Scan3dCoordinateOffset"
127     camera.remote_device.node_map.ChunkEnable.value = True
128
129     camera.remote_device.node_map.Scan3dCoordinateSelector.value = "CoordinateC"
130     camera.remote_device.node_map.Scan3dCoordinateScale.value = 0.1
131     camera.remote_device.node_map.Scan3dCoordinateOffset.value = -15000.0
132
133     # setup plot
134     gridsize = (1, 2)
135     fig = plt.figure()
136     axOrgSurf = plt.subplot2grid(gridsize, (0, 0))
137     axRWSurf = plt.subplot2grid(gridsize, (0, 1))
138     fig.suptitle('Measurements')
139     axOrgSurf.set_title('Original Surface')
140     imOrgSurf = axOrgSurf.imshow(np.ones([542, 512]))
141     axRWSurf.set_title('Real-World Surface')
142     imRWSurf = axRWSurf.imshow(np.ones([542, 512]))
143     plt.show(block=False)
144     plt.draw()
145     plt.pause(0.1)
146
147     # acquire data
148     print('start acquisition')
149     camera.start()
150     for itr in range(0, 5):
151         camera.remote_device.node_map.TriggerSelector.value = "FrameStart"
152         camera.remote_device.node_map.TriggerSoftware.execute()
153
154         buffer = camera.fetch(timeout=10.0)
155
156         camera.remote_device.node_map.ChunkScan3dCoordinateSelector.value =
157         ↪ "CoordinateC"
158         coordCScale = camera.remote_device.node_map.ChunkScan3dCoordinateScale.
159         ↪ value
160         coordCOffset = camera.remote_device.node_map.ChunkScan3dCoordinateOffset.
161         ↪ value
162         print(f"ChunkScan3dCoordinateScale: {coordCScale}, □
163         ↪ ChunkScan3dCoordinateOffset: {coordCOffset}")
164
165         height = buffer.payload.components[0].height
166         width = buffer.payload.components[0].width
167         orgSurf = buffer.payload.components[0].data.reshape(height, width)

```

(continues on next page)

(continued from previous page)

```

165     orgSurf = np.copy(orgSurf)
166     rwSurf = orgSurf * coordCScale + coordCOffset
167     orgSurfMean = np.mean(orgSurf)
168     rwSurfMean = np.mean(rwSurf)
169     print(f"Surface Mean Values: Original {orgSurfMean}, Real-Word:
→ {rwSurfMean}")
170
171     imOrgSurf.set_data(orgSurf)
172     imOrgSurf.set_clim(vmin=np.min(orgSurf), vmax=np.max(orgSurf))
173     imOrgSurf.set_extent((0, width, height, 0))
174
175     imRWSurf.set_data(rwSurf)
176     imRWSurf.set_clim(vmin=np.min(rwSurf), vmax=np.max(rwSurf))
177     imRWSurf.set_extent((0, width, height, 0))
178
179     plt.draw()
180     plt.pause(0.1)
181
182     buffer.queue()
183     camera.stop()
184     except Exception as error:
185         print(error)
186
187     camera.destroy()
188     print("SCRIPT END")

```

2.5 Document History

Version	Date	Changes
15.2.0.0	19-05-2022	initial version

TN_15.3.0.1: COLLECTING AND INTERPRETING MONITORING INFORMATION OF GEN4 DEVICES

3.1 Context and Challenge: Finding Root Causes of Non-Systematic Issues

HeliiInspect H8, H9, or heliCam C4 devices may suddenly, sporadically, or gradually fail or deliver poor quality data. Hardware defects or environmental issues, including interactions on their interfaces, are possible root causes.

The devices and software tools do, for the time being, not provide accessible log files or interactive monitoring of internal or external operating conditions. Finding root causes of such issues, therefore, requires high debugging efforts.

3.2 Solution: Client-Based System Monitoring

Debugging of issues is simplified or made possible in the first place by collecting and possibly monitoring status information from the device. This should be implemented in the client (consumer) application that controls the heliCam/heliInspect device.

The tables below point at useful status information. Client applications read it through the standard camera interface at the configuration stage, after performing a measurement, or in case of an exception. Depending on the type of information and on user requirements, the application may implement monitoring and/or logging.

3.2.1 Permanently Available Status Information

The client application can read the device and status information listed below at any time as long as it has an open connection to the device.

Feature	Value [expected / healthy]
DeviceFirmwareVersion	camera embedded SW version (format e.g. 1.x.y)
DeviceFPGAFwVersion	camera fpga firmware version in cryptic format
HeliDriverFirmwareVersion	heliDriver firmware version (format: e.g. 1.x)
HeliDriverStatus	status of the heliDriver in cryptic format
DeviceTemperature [DeviceTemperatureSelector= <i>Processor</i>]	< 80
DeviceMonitorActualValue [DeviceMonitorSelector = <i>Vdd-Voltage</i>]	between 21.5 and 26.5
DeviceMonitorActualValue [DeviceMonitorSelector = <i>Vdd-Current</i>]	< 1.5
DeviceMonitorActualValue [DeviceMonitorSelector= <i>UserVdd-Voltage</i>]	between 21.5 and 26.5
DeviceMonitorActualValue [DeviceMonitorSelector= <i>UserVdd-Current</i>]	< 0.15

Note: Refer to the feature description document for data types and detailed information about the features mentioned.

Note: Some features are sensitive to values of selector features. Refer to the programmer's manual for more information on the concept.

Note: Providing logged information in support requests to heliotis is very helpful. In particular, please contact heliotis for interpreting information in cryptic format.

3.2.2 Status Information Relevant after Configuration

The status information below is readable any time but its content is sensitive to configuration features. The client application may log it after configuring the device.

Feature	Value [expected / healthy]
ActualVerticalSpacing FrameRate	near TargetVerticalSpacing
	H8,H9,C4 (542x512 pixel) devices: < 4000 H8M, H9M, C4M (1.1M pixel) devices: < 1000
RecordingNFrame TriggerPosition	approximately: $\frac{ScanRange}{ActualVerticalSpacing}$ defined by ScanMode , ScanPosition , ScanRange
LightBrightness [LightControllerSelector = <i>Light-Controller0</i>]	10% to 100%

Note: FrameRate maximum values are approximate. The exact maximum depends on further settings.

3.2.3 Status Information of the Acquisition Engine

The following status information represents the internal state of the acquisition engine. In case of a timeout during recording, the client application could find out why the acquisition engine got stuck.

Feature	Value
AcquisitionStatus [AcquisitionStatusSelector = <i>FrameTriggerWait</i>]	<i>True</i> : state active, <i>False</i> : state inactive
AcquisitionStatus [AcquisitionStatusSelector = <i>RecordingTriggerWait</i>]	<i>True</i> : state active, <i>False</i> : state inactive
AcquisitionStatus [AcquisitionStatusSelector = <i>FrameActive</i>]	<i>True</i> : state active, <i>False</i> : state inactive
AcquisitionStatus [AcquisitionStatusSelector = <i>RecordingActive</i>]	<i>True</i> : state active, <i>False</i> : state inactive
TransferStatus [TransferStatusSelector = <i>Streaming</i>]	<i>True</i> : state active, <i>False</i> : state inactive
StageStatus [StageStatusSelector = <i>InMotion</i>]	<i>True</i> : state active, <i>False</i> : state inactive

Note: All possible acquisition engine states are documented in the appendix of the feature description.

3.2.4 Status Information Related to Measurement Data

The following status information is related to measurement data. It should be read after a successful measurement and always relates to the latest measurement.

Feature	Value (expected / healthy)
LightCurrentMeasured [LightControllerSelector = <i>LightController0</i>]	$LightCurrentRating[.] * \frac{LightBrightness[.]}{100\%}$
ChunkZTagCount	equal to RecordingNFrame
ChunkZTagValue [ChunkZTagSelector = <i>1</i>]	$\frac{TriggerPosition}{EncoderResolution[Camera]}$
ChunkProcessingString	(.json string) dump from on-camera data processing
ChunkConfigString	(.json string) dump of camera configuration

Note: Make sure the illumination is enabled when reading the value of **LightCurrentMeasured**. Typical values for LED current rating are 1A (H8) or 2A (H9).

Much of the measurement-related information are transmitted as GenTL ChunkData along with measurement data. The host application must enable transmission by setting the following configuration features already before the measurement, e.g. when initializing the camera:

Listing 3.1: Enable relevant chunk data

```

ChunkModeActive = True
ChunkSelector = "ZTagCount"
ChunkEnable = True
ChunkSelector = "ZTagValue"
ChunkEnable = True
ChunkSelector = "ProcessingString"
ChunkEnable = True
ChunkSelector = "ConfigString"
ChunkEnable = True

```

Note: Some of the debugging chunk data like *ConfigurationString* could significantly increase the measurement cycle time. General enabling is not recommended.

3.3 Document History

Version	Date	Changes	Responsible
15.3.0.0	14-11-2022	initial version	cl
15.3.0.1	01-03-2024	review/correction	sm

TN_19.1.0.1: USERSET - A SHORT INTRODUCTION

4.1 Context and Challenge: Storing gen4 Device Configurations

Heliotis Gen4 devices offer highly customizable configurations through device features. In some cases, one may not want to specify the entire configuration for an inspection task in the control application.

Additionally, some configurations are explored in a lab environment before being applied in production. The standard approach is to implement the new configuration directly in the user application. However, in certain situations, modifying the user application is not feasible/permitted.

In these cases, it becomes necessary to store specific configurations directly on the device.

4.2 Solution: UserSet

To address these scenarios, Gen4 devices support the *UserSet* functionality, as defined by GenICam and described in the Standard Feature Naming Convention (SFNC) of GenICam¹. This technical note outlines various use cases and procedures for configuring Heliotis Gen4 devices through UserSets.

Heliotis devices offer two custom UserSets: *UserSet0* and *UserSet1*, which serve as independent storage locations for full device configurations. A third option, *Default*, is available through the *UserSetSelector* feature. The *Default* option is read-only and represents the factory default configuration.

The primary UserSet operations include:

- *Store* – Saves the current configuration to the selected storage location on the device.
- *Load* – Loads a previously stored configuration from the selected storage location.

Additionally, users can define which UserSet (*UserSet0*, *UserSet1*, or *Default*) should automatically load upon device boot-up or reset.

4.2.1 Storing (or Creating) a UserSet

To *store* a UserSet, an application or script must connect to the Gen4 device and configure it appropriately. Matching feature values can be determined using a GenICam Viewer or the Heliotis-specific *heliViewer*. Further recommendations may be provided by Heliotis.

Important: A UserSet must be stored while the device is in configuration mode (i.e. not during an acquisition) and before the device connection is closed (which triggers a reset).

1. First, select the desired UserSet using *UserSetSelector*.
 2. Then, execute the corresponding save operation (e.g., *UserSetSave*).
-

¹ EMVA GenICam SFNC: https://www.emva.org/wp-content/uploads/GenICam_SFNC_v2_7.pdf

Listing 4.1: Store a UserSet

```
UserSetSelector = "UserSet0"  
UserSetSave.execute()
```

4.2.2 Loading a UserSet

To apply a previously stored UserSet, the application must execute the Load operation after establishing a device connection. This process loads the full camera configuration, overwriting any previously set feature values.

Listing 4.2: Load a UserSet

```
UserSetSelector = "UserSet0"  
UserSetLoad.execute()
```

4.2.3 Default UserSet

Gen4 devices support configuring a *Default UserSet* that loads automatically upon boot-up, reset, or reconnection. This allows users to define default values for the device.

The feature name for configuring the *Default UserSet* is `UserSetDefault`. Unlike most other features, `UserSetDefault` is non-volatile, meaning its value remains unchanged even after device resets, reconnections, or firmware updates.

Available options for `UserSetDefault`:

- UserSet0
- UserSet1
- Default (factory default setting)

Listing 4.3: Configure the default UserSet

```
UserSetDefault = "UserSet0"
```

4.2.4 File Transfer

Users may need to transfer a stored *UserSet* from one device to another. Gen4 devices support *UserSet* transfers through the *FileAccess* interface, as defined by GenICam.

Most GenICam-compatible applications support file transfer. Additionally, Heliotis provides *updateTool*, which facilitates UserSet transfers to and from devices.

Within *updateTool*, users can access the *Manage Files* dialog to select a device and define the desired UserSet through *FileSelector*. The transfer process from and to the file specified in *Local File* can then be initiated using *Write to Device* or *Read from Device*.

See also (see [Check File Transfer](#))

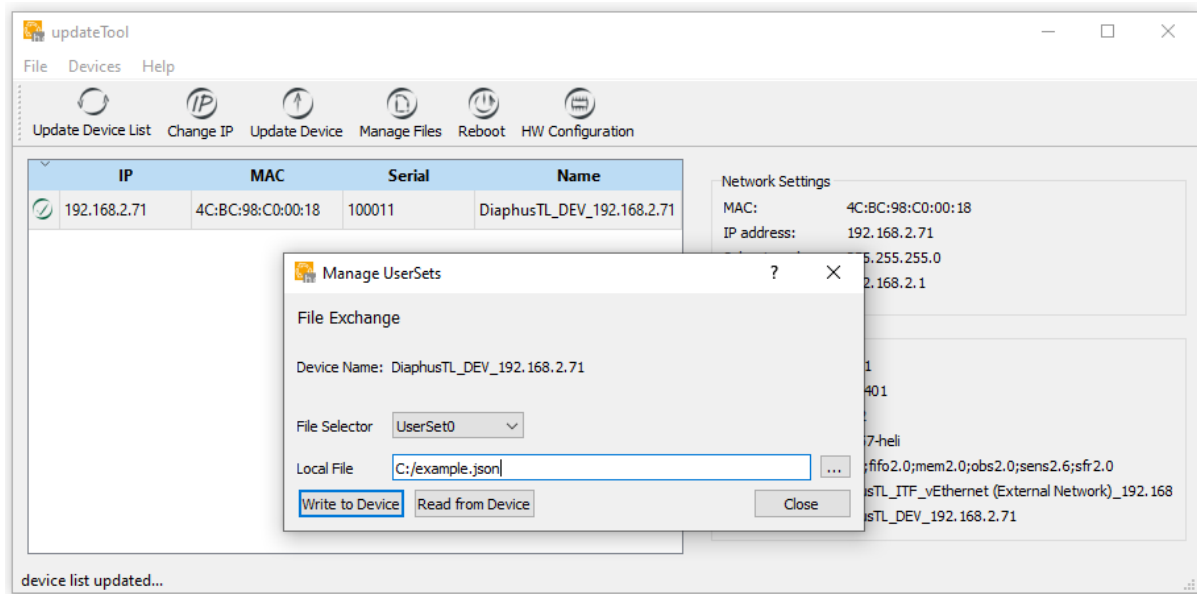


Fig. 4.1: File Transfer support in the updateTool

Limitations

1. The target device must have the exact same hardware configuration as the source device, since certain features depend on hardware compatibility.
 - Example: External Xenax stages require IP address definitions that other stage types do not support.
2. The target device must have firmware that is the same or newer than the source device's firmware.
 - While older UserSets can be migrated to newer firmware versions during file transfer, newer UserSets cannot be read by older firmware.
3. The File Transfer process does not provide an error notification when an unsupported UserSet is sent to a device.
 - see [Check File Transfer](#)

4.2.5 Additional Features

AutomatedStageInit

When using heliInspect, the initial configuration after a power-up requires executing the StageInit command to initialize the stage. This procedure is not automated by default since additional configuration steps (e.g., defining an IP address) may be required.

With the UserSet mechanism, the full configuration - including IP address - becomes available at boot time, allowing automated stage initialization. Users can enable this feature by setting `AutomatedStageInit = true`.

Check File Transfer

The File Transfer implementation does not generate an error message if an unsupported UserSet is transferred to a device.

To verify whether the transfer was successful, users can access *log* information using the *File Manager* in *updateTool*.

1. Select Read from Device to retrieve the logs.
2. Store the logs as a .tar.gz file on the host computer.
3. Open the log file to check for error messages.

Listing 4.4: Example of an Error message when the UserSet transfer fails

```
[185582.447516]      Global 1956635728      ERROR      Failed to update UserSet0:␣
↪ Could not convert file /data/heliService/userSets/userSet0.json
```

4.3 Document History

Version	Date	Changes	Responsible
19.1.0.0	22-11-2024	initial version	sm
19.1.0.1	12-06-2025	editing	bm

TN_19.10.0.1: CONFIGURING OFFSET (FPN) CORRECTION

5.1 Challenge: FPN Removal as a Necessary Pre-Processing Step

Raw measurement data is subject to a constant, pixel-dependent offset called fixed pattern noise (FPN), see [Fig. 5.1](#). Knowing and removing each pixel's FPN is required prior to subsequent on-camera processing (« *H8 Programmer's Guide*, chapter *Signal Processing*). FPN can be measured by averaging raw data across frames void of any interference signal. How you can ensure the FPN correction is accurate is explained in this technical note.

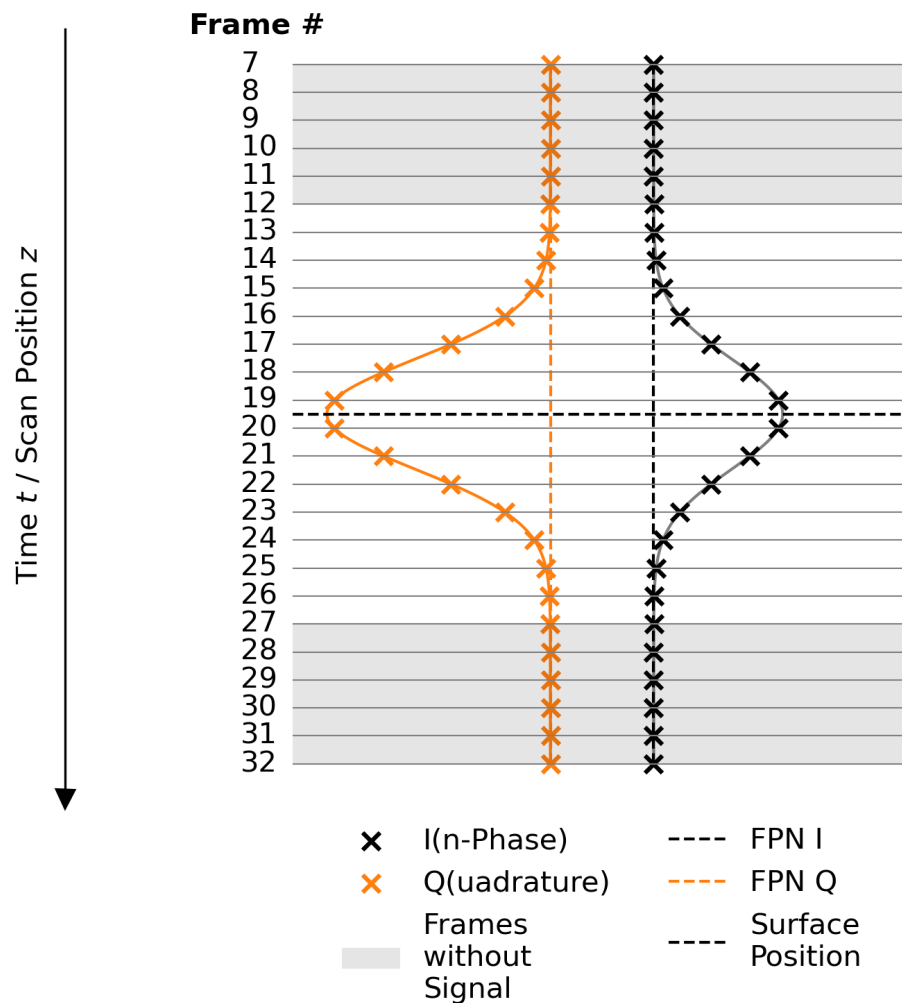


Fig. 5.1: Schematic, Single-Pixel Raw WLI Signal in heliInspect Devices are Offset by FPN

In every scan with a heliInspect, a stack of frame pairs is recorded, containing the signal's In-Phase, I , and Quadrature, Q , components respectively. I and Q each has a pixel-dependent semi-random offset called FPN that is constant across frames. The signal processing implemented on heliInspect devices measures FPN on both components individually for ensuing subtraction. It does so by pixel-wise averaging of a configurable subset of frames assuming that these do not contain interference signal.

5.2 Solution: Configure a Frame Range for Averaging Sufficiently Far from the Surface

The configurable number of frames at either end of the scan range can be used to determine FPN. You may thus set this range of frames to be sufficiently far from the surface position in the frame stack using camera features.

5.3 Implementation

The feature `FPNCorrection` allows you to choose a zone of frames for FPN correction either at the beginning (`AverageFirstFrames`) or the end (`AverageLastFrames`) of the scanned volume.

`FPNCorrectionNFrames` sets the depth of the zone (in units of frames) used for FPN calculation.

Listing 5.1: Configure FPN Correction Using 8 Last Frames

```
FPNCorrection = "AverageLastFrames"
FPNCorrectionNFrames = 8
```

Note: Valid settings of `FPNCorrectionNFrames` are 1, 2, 4, 8, 16, 32 & 64. Values below 8 may affect the repeatability of topology results since additional random noise gets insufficiently averaged.

Note: The selection between `AverageFirstFrames` and `AverageLastFrames` refers to the time axis. Whether these are the uppermost or lowest frames depends on the scan direction configured determined the feature `ScanMode`.

Important: Unsound and or even erratic results will arise if the measured sample surface is within or close to the specified range of frames used for FPN calculation.

The center of the scan interval is given by `ScanPosition` and its full length by `ScanRange` in physical units. `RecordingNFrames` returns the number of frames recorded over this scan range.

The scan range effectively available for the surface measurement is given by:

$$scanRange - actualVerticalSpacing \cdot FPNCorrectionNFrames$$

5.4 Document History

Version	Date	Changes	Responsible
19.10.0.0	07-04-2025	Initial version	
19.10.0.1	12-06-2025	Review	bm

TN_19.2.0.1: CONFIGURING THE OPTIC TYPE

6.1 Context: Variety of Optics Types for heliInspect H8/H9

heliInspect H8 and H9 devices are available with different optics types, implementing different magnifications and/or interferometer designs. For optimum device performance, a group of camera features requires settings specific to the optic type installed on the device. These features can be written individually or loaded from predefined configurations.

6.2 Challenge: Correctly Configure heliInspect H8/H9 for an Optics Type

The magnification, the numerical aperture and the effective wavelength depend on the optic type used. Also, the peak shape of the raw interferometric signal is altered. As a consequence, the vertical sample spacing of a scan needs to be adjusted in order to sample the signal appropriately. The programmers guide includes recommendations for the `TargetVerticalSpacing` values for different optic types. As a rule of thumb, the vertical spacing needs to be lower for higher magnifications.

6.3 Solution: Load Predefined Configurations using the OpticType Feature

The camera feature `OpticType` allows to load predefined, recommended, configurations for `Magnification`, `NumericalAperture`, `WavelengthCorrection` and `TargetVerticalSpacing`. Consequently, any changes of these features written before the `OpticType` feature is updated will be lost. When changing any of these features individually after setting the `OpticType` feature, the `OpticType` will take the value `Custom`.

Note: The `EffectiveWavelength` value predefined in `OpticType` configurations reflects nominal optic properties. If production variation and temperature variation effects are to be covered, the fine-tuning of `WavelengthCorrection` is required. The fine tuned value must be written after the `OpticType` feature.

Note: The values of `TargetVerticalSpacing` defined in `OpticType` configuration strike a trade-off between accuracy (improves with decreasing spacing) and measurement cycle time (maximum available scan speed increases and amount of data decreases with increasing spacing). Try half of the predefined value for optimized accuracy or twice the predefined value for optimized cycle time, respectively.

6.4 Document History

Version	Date	Changes	Responsible
19.2.0.1	19-02-2025	initial version	ls

TN_19.3.0.1: REDUCING VIBRATION-INDUCED WLI AMPLITUDE LOSS

7.1 Context: Ideal Scanning of Sample-to-Optic Distance

The WLI measurement principle relies on vertical scanning of the distance between the sample and the interferometer optic. Optical interference signals are formed at a *signal frequency* defined by the vertical scan velocity and the illumination wavelength.

Signal processing in heliInspect gen4 devices detects interference amplitude from these signals at an expected constant *detection frequency* based on the values of the ScanSpeed, NominalWavelength, and WavelengthCorrection features.

Ideally, the actual *signal frequency* is constant and matches the *detection frequency*.

7.2 Challenge: Amplitude Drop due to Velocity Deviations

Variations of the actual *signal frequency* arise from scanning the distance between the sample and the interferometer optic at non-constant velocity. Deviations from constant velocity are caused by either uneven scan motion or mechanical vibrations of the structure between the sample surface and the heliInspect device.

If the actual *signal frequency* deviates from the *detection frequency*, the detected amplitude drops. The signal-to-noise ratio of the detected interference signal is, therefore, reduced. Consequently, the 3D surface data is affected by increased random noise (repeatability) and vertical distortion (inaccuracy).

The following factors determine the quantitative impact of such *velocity deviations* on the detected amplitude in gen4 heliInspect devices:

- relative amount of *velocity deviation* $((v_{\text{sample-optic}} - \text{ScanSpeed}) / \text{ScanSpeed})$
- vertical signal *observation interval* of a frame (by default, this is close to the value of ActualVerticalSpacing)

The larger the product of the above two terms, the worse the amplitude drop.

If mechanical vibrations are at the origin of velocity deviations, amplitude loss is observed as a sporadic artifact: For flat horizontal sample surfaces amplitude may drop in the full FOV of random measurement iterations. For samples with larger topographic range, random height intervals (z intervals) might be affected (“dark isolines”).

7.3 Fundamentals: Correct Wavelength Setting and Avoidance of Velocity Variations

To avoid systematic mismatch between the actual interference waveform *signal frequency* and the *detection frequency*, make sure the value of the `EffectiveWavelength` feature matches the actual wavelength.

Note: Refer to technical note TN 19.2 (*Configuring the Optic Type*) for setting `EffectiveWavelength` using presets.

Note: Refer to the programmer's guide chapter on wavelength tuning for fine-tuning `EffectiveWavelength` based on actual measurement data.

The most effective and systematic way of avoiding amplitude drop is avoiding *velocity deviations* at the potential root cause(s):

- Implement a stiff and short mechanical link between the sample and the attachment point of the heliIn-spect device.
- Avoid excitation of mechanical vibrations in the mechanical setup (avoid inducing shocks, or high accelerations. Don't couple vibrating subsystems to the metrology setup)
- If using a third-party vertical scan stage, optimize stage control for constant scan velocity. As a rule of thumb, relative scan *velocity deviation* should be under 3% (considering a 0Hz to 5kHz band).

Note: External (third party) stages with high-accuracy encoders and programmable encoder output increment can be configured and used as position-based sources of external reference trigger. In that case, the impact of the scanner's speed deviation is overcome entirely. Impact of sample vibration remains.

7.4 Parametric Solution: Increase ScanSpeed and Decrease VerticalSpacing for Improved Robustness

The impact of *velocity deviations* is reduced by the following choices of parametrization:

- Increasing the value of the `ScanSpeed` feature reduces the impact of vibrational *velocity deviation*.
- Decreasing the vertical *observation interval* (i.e decreasing `ActualVerticalSpacing`) reduces the impact of any *velocity deviation*.

This approach is limited by internal operation details of gen4 devices: A stack of *internal frames* is recorded and stored while the device vertically scans through the scan range. One *internal frame* is recorded and stored each distance increment of `ActualVerticalSpacing`. The frame count of the stack is thus equal to `ScanRange` divided by `ActualVerticalSpacing`.

The above-described parametric optimization is, hence, subject to the following limits:

- The maximum storable internal frame count is limited. Maximum scan range is, therefore, limited to the product of `ActualVerticalSpacing` * maximum frame count. Decreasing vertical spacing might shrink the maximum scan range below the application requirements.
- The internal rate of recording and storing frames is given by `ScanSpeed / ActualVerticalSpacing`. This rate is subject to a maximum, too. This criterion limits the approach of simultaneously increasing `ScanSpeed` and decreasing `ActualVerticalSpacing`.

After the scan, gen4 devices process the stack of stored frames for extracting 3D topography. The duration of processing is proportional to the frame count. As decreasing `ActualVerticalSpacing` increases the internal frame count, this optimization comes at a cost of measurement latency and, possibly, cycle time.

7.5 Extended Parametric Solution: Low VerticalSpreading Enables Very Short Observation Interval

The degree of *exposure time spreading* is configurable in gen4 devices. The concept refers to choosing whether interference signals are observed and detected continuously (fully spread) over the duration of a frame or during a confined shorter interval (less spread). The latter choice enables very short vertical *observation interval* without entailing the above-mentioned issues of high internal frame rate and high frame count.

The figure below visualizes the implementation of configurable *exposure time spreading* and the close interaction with exposure time control (usage of the `ExposureRatio` feature).

By default (when `TargetVerticalSpreading` = 100%), the exposure time of a frame is spread over the entire frame.

If `ExposureRatio` = 1.0 the frame's exposure time is essentially equal to the frame duration, and the vertical *observation interval* is, hence, equal to the frame spacing (`ActualVerticalSpacing`).

If `ExposureRatio` is set to a value lower than 1.0, the frame exposure time is shorter than the frame duration. In that case, the exposure will be pulse with modulated (intermittingly turned on and off several times per frame) with a duty cycle defined by `ExposureRatio`. However, the *observation interval* will still be as long as the frame spacing.

By setting `TargetVerticalSpreading` to a value below 100%, the user reduces the *observation interval*. As shown below, the *observation interval* shortening is, internally, achieved by reducing the number of cycles. The device maintains constant overall exposure duration, and hence WLI amplitude, by compensating with a duty cycle increase.

Decreasing `TargetVerticalSpreading`, therefore, effectively reduces the impact of vibrations and uneven scan motion.

Note: As the visualization hints, the actual degree of exposure spreading applied is subject to some limitations and quantization. The user specifies a target percentage in the `TargetVerticalSpacing` feature. Reducing `TargetVerticalSpreading` below `ExposureRatio` has no additional effect. In that case, `ActualVerticalSpreading` will be set to the value of $100\% * \text{ExposureRatio}$ internally. Further, The vertical *observation interval* cannot be reduced below the wavelength value. In that case `ActualVerticalSpreading` will be set to $100\% * \text{EffectiveWavelength} / \text{ActualVerticalSpacing}$ internally.

Note: Unfortunately, reducing `TargetVerticalSpreading` might not be a practical solution for high-magnification (20x, 50x) optics. These require low `ActualVerticalSpacing` and, depending on illumination type and sample reflectance, high `ExposureRatio`.

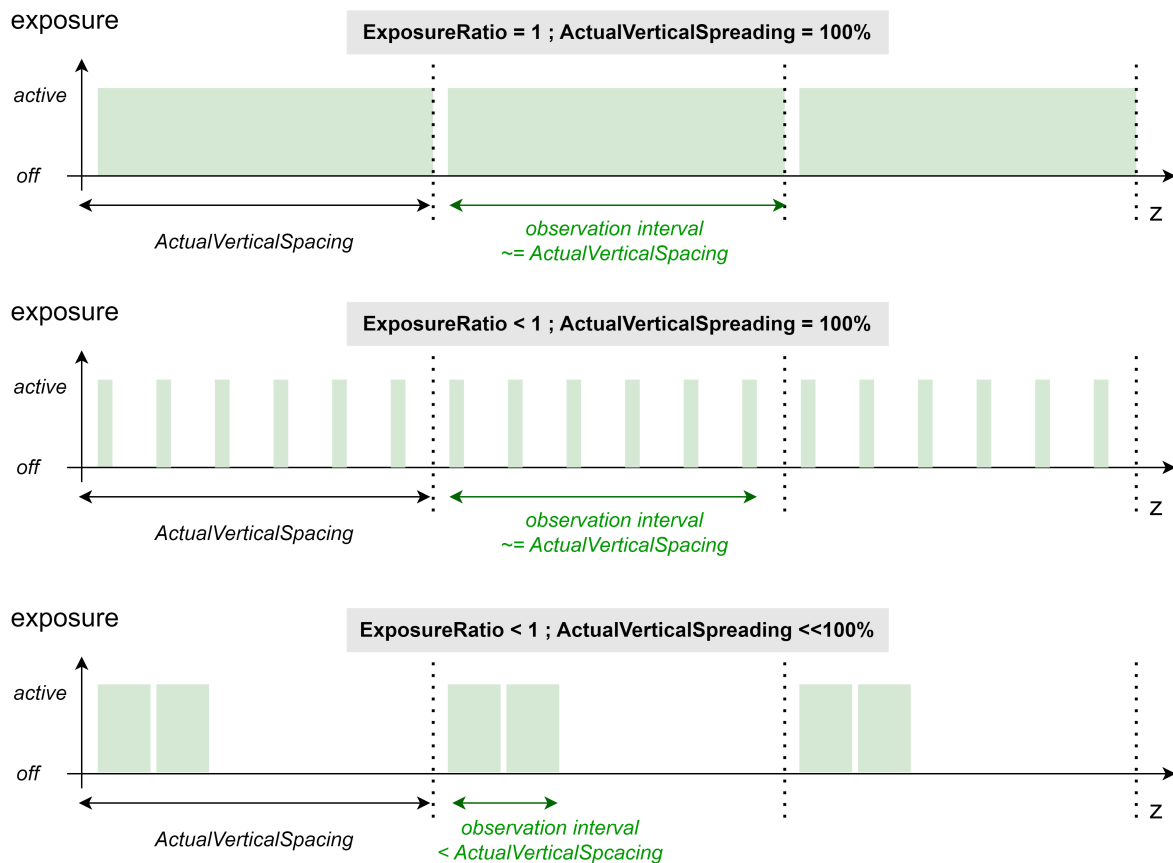


Fig. 7.1: Exposure diagrams as a function of ActualVerticalSpreading and ExposureRatio (conceptual visualization)

7.6 Practical Implementation

As a starting point, use camera parameters that fulfill the prerequisite of matched wavelength setting and provide satisfactory data (while possibly suffering from sporadic velocity mismatch artifacts). Implement the optimization following the sequence below:

- Set `LightBrightness` to a value of 100%.
- Increase `TargetVerticalSpacing` to a large but reasonable value: E.g. use the preset `TargetVerticalSpacing` value as loaded when setting the `OpticType` feature to the value representing the optic hardware installed. Or even 1.5x this value.
- (Iteratively) increase `ScanSpeed` to the maximum functional value.
- Iteratively decrease `ExposureRatio` to the lowest value providing reasonable amplitude. Stop decreasing `ExposureRatio` at a value of 2% and decrease `LightBrightness` instead.
- Set `TargetVerticalSpreading` to a value of 1% for shortest possible *observation interval*.
- Validate that amplitude is stable and sufficient over measurement repetitions in the entire FOV.

Note: In presence of significant vibrations, minimizing ActualVerticalSpreading provides best results. In exceptional cases (only moderate vibrations and low sample reflectance) optimum values of TargetVerticalSpreading are anywhere between 0% and 100%.

7.7 Document History

Version	Date	Changes	Responsible
19.3.0.1	25-03-2025	initial version	cl

TN_19.4.0.1: SCAN MOTION MONITORING (ZTAG-READING)

8.1 Challenge: Monitoring Scan Motion Profile

For precise WLI measurements, it is essential that the measuring head moves at a well-controlled, constant speed with respect to the sample. Departures from this ideal case can occur due to sample movement (see [TN_19.3.0.1: Reducing Vibration-Induced WLI Amplitude Loss](#)) or irregular scan stage motion. Detecting the latter for monitoring and optimization purposes is the subject of this technical note.

helioInspect devices keeps track of the motor stage position at the centre of each camera frame, see [Fig. 8.1](#). (In most operating modes, a surface and a reflectance image is extracted from the ensemble of these frames behind the scene.) The motor feedback, or more precisely the *motor encoder values*, can be retrieved as *zTags* from the same buffer as other measurements data.

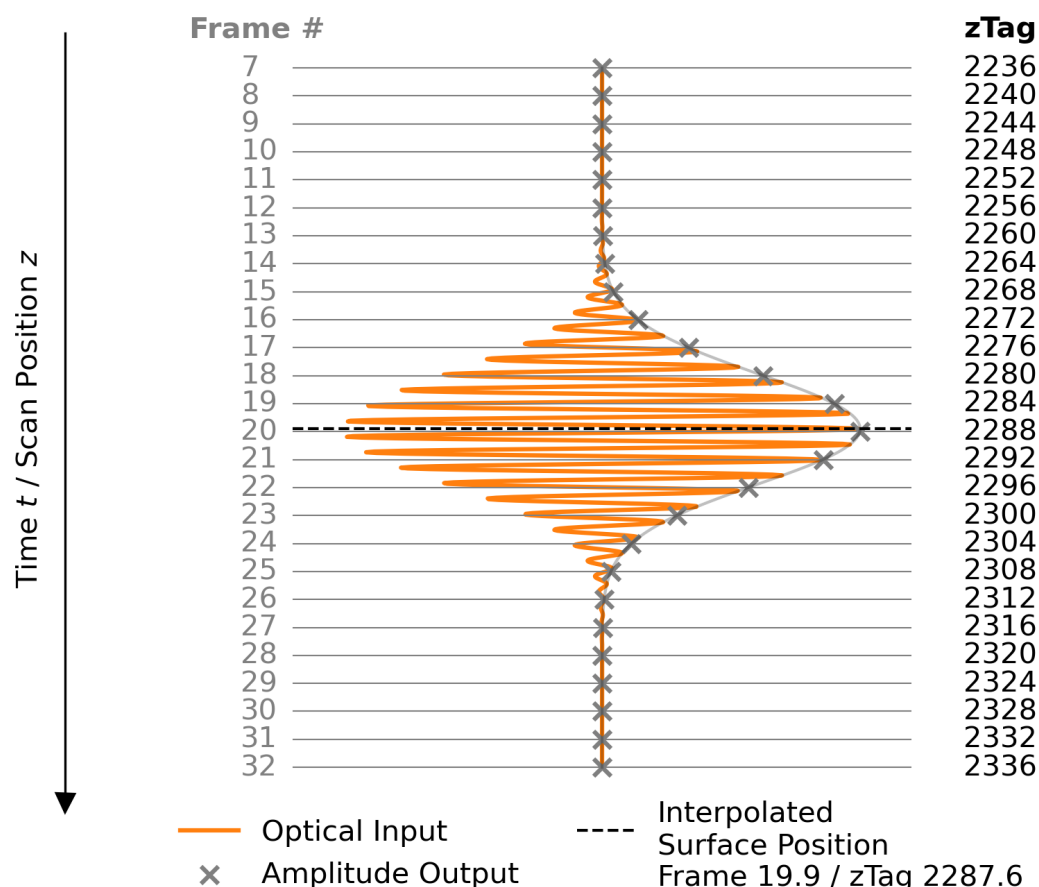


Fig. 8.1: Illustration of heliInspect zTag Output Alongside In-Pixel Signal Processing

In every scan with a heliInspect, a stack of frames is recorded. Each individual frame represents the instantaneous envelope (and phase shift) of a high-frequency input signal in a spatially resolved manner. The camera also extracts the central motor position, or zTag, in terms of (integer) motor encoder increments for each frame. In most cases, these data are processed directly on the camera to determine the interpolated peak position, which corresponds to the surface position in every pixel. Other possible outputs of such processed extraction modes are the value of the envelope or the phase shift at the signal peak.

8.2 Implementation

To retrieve zTags, the camera must be configured to write them to the buffer containing measurement data as part of the so-called *chunk* components. Here are the corresponding instructions in the form of pseudo code.

Listing 8.1: Enable and Configure Chunk Data

```
# enable chunk data
ChunkModeActive = True
ChunkSelector = "ZTagCount"
ChunkEnable = True
# enable zTag related chunk data
ChunkSelector = "ZTagValue"
ChunkEnable = True
```

Note: This piece of code must be executed in the camera's configuration mode prior to the start of the data acquisition.

After the measurement, each frame's zTag can be retrieved by means of a loop.

Listing 8.2: Retrieving Device Data

```
# accessing zTags included in chunk data (part of acquired buffer)
nofZTags = ChunkZTagCount

zTags = array of size nofZTags
# loop over all available zTag values (index from 0 to nofZTags-1)
  ChunkZTagSelector = index
  zTags[index] = ChunkZTagValue
```

Note: This piece of code must be executed in the camera's acquisition after the buffer carrying data has been fetched.

Note: To properly terminate the loop, the number of zTags - or equivalently frames - is read directly from the buffer.

To convert the stage position of the i^{th} frame expressed as a zTag, $zTag[i]$, to physical units, they must be multiplied by the encoder resolution, Δ_{zTag} .

$$z[i] = zTag[i] \cdot \Delta_{zTag} \quad (8.1)$$

The encoder resolution can be read from the camera using the feature `EncoderResolution`.

Listing 8.3: Retrieving Encoder Resolution

```
# read encoder resolution in nm/tick
encoderRes = EncoderResolution
```

8.3 Interpretation

An example of one adequate and one inadequate outcome of the stage position monitoring are shown in Fig. 8.2.

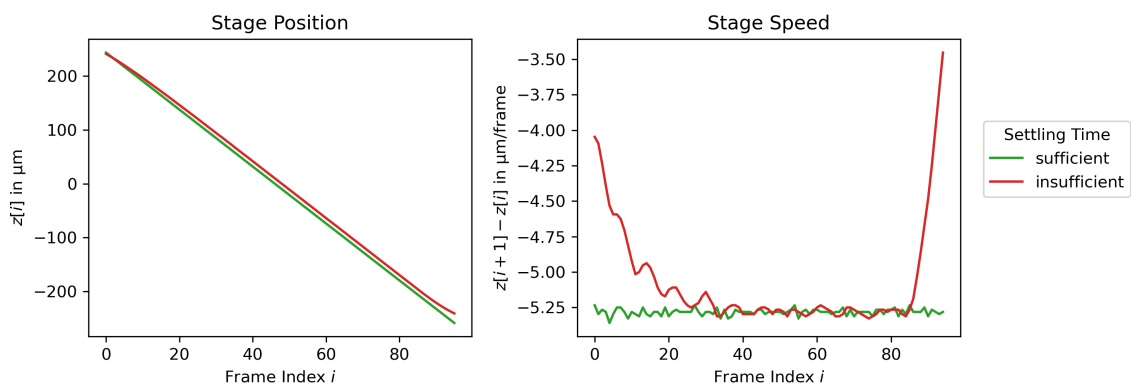


Fig. 8.2: Satisfactory and Unsatisfactory Motion Profiles Measured via zTags and Converted to Physical Units In the former case, the scan parameters were selected to produce a linear, controlled stage movement. In the latter case a high ScanSpeed and a small AccelerationRamp were set. As a result, the scan speed is not yet stable during image acquisition, resulting in a non-linear movement.

8.4 Document History

Version	Date	Changes	Responsible
19.4.0.0	07-04-2025	Initial version	
19.4.0.1	11-06-2025	Review	bm

TN_19.9.0.0: FIRST/LAST SURFACE EXTRACTION

9.1 Challenge: Interference Envelope of Transparent Samples

For a single scan of transparent and semi-transparent samples a pixel's interference envelope comprises one peak per reflective interface in the scan range (» Fig. 9.1).

A single surface is, hence, not defined unambiguously. However, the application may either require measuring the first surface consistently or the last surface consistently.

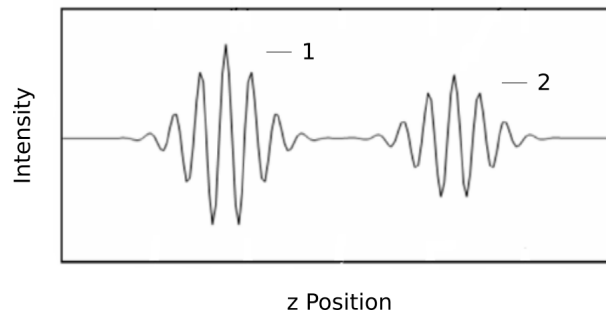


Fig. 9.1: Sample comprising a transparent layer: Typical intensity waveform of a pixel plotted in the z-direction with (1) first surface and (2) second surface

9.2 Solution: Amplitude Peak Detection and Thresholding

- Use `PeakDetectionMode` to distinguish first and last occurrences of amplitude maxima along `z` in scan direction. Relevant entries are: `FirstMax`, `LastMax`, `FilteredFirstMax`, `FilteredLastMax` (The latter two use amplitude peak filtering. For more details refer to » vertical-filtering-amplitude-peak).

Note: `PeakDetectionMode` configures the algorithm used for coarse surface detection. It has no influence when `Scan3dExtractionMethod` is `rawIQ` or `rawAmp`.

- If selecting `FilteredFirstMax` or `FilteredLastMax`, define a value for `PeakDetectionFilterWidth`.
- Define a suitable amplitude threshold value to enable the device to separate noise from the relevant signal and write it to `PeakDetectionThreshold`. Iteratively optimize the threshold value. Noise-induced surface outliers appear at random height if the value is too low. No surface is detected (pixel's `z` data set to end of scanned range) if the threshold value is too high.

Note: A good starting point for `PeakDetectionThreshold` can be found by first measuring in `rawAmp` mode and selecting an amplitude value between the noise floor and the envelope peak of interest. A good

starting point for the PeakDetectionFilterWidth is the width of the signal amplitude peak. Filtering slightly reduces peak amplitude. Reduce the value of PeakDetectionThreshold accordingly.

9.3 Tips for Narrow Transparent Layers

When peak separation is a small multiple of peak width:

- Use minimum vertical spacing (TargetVerticalSpacing equal to wavelength if BackgroundSuppression = 'off', 1.5*wavelength otherwise)
- Use unfiltered surface detection if peak amplitude is consistently well above noise level (FirstMax or LastMax).
- When using filtered surface detection (FilteredFirstMax, FilteredLastMax), use PeakDetectionFilterWidth values lower than the peak separation (in unit of frames)
- Use ExtSimpMaxHwin values lower than the peak separation (in units of frames). ExtSimpMaxHwin defines the vertical size of the data window (around the detected peak, in unit of frames) used for (fine) surface extraction. If using a larger data window size, the peak is not properly isolated and adjacent peaks may alter the fine surface result.

9.4 Implementation

```
# Enable first surface detection across the a-scan.
PeakDetectionMode = "FirstMax"

# Configure amplitude threshold
PeakDetectionThreshold = 84
```

9.5 Document History

Version	Date	Changes	Responsible
19.9.0.0	07-04-2025	Initial version	cl

TN_51.1.0.0: HELIOTIS GEN4 DEVICE FIELDBUS REFERENCE FOR D3

10.1 Introduction

The Heliotis gen4 device family (heliCam™ C4, heliInspect™ H8/H9) offers optional fieldbus capability in combination with a fieldbus module build into the heliDriver™ D3A. Heliotis currently offers modules supporting either PROFINET RT or EtherCAT®. This capability enables the gen4 device to be integrated into industrial networks for data exchange and control. The heliDriver poses as a fieldbus device or slave and can be controlled by a programmable logic controller (PLC). To integrate the device into the fieldbus, both the device and the PLC that controls it need to be configured correctly. This document serves as a reference for the *configuration of both sides* with the use of features on the camera and the description file (ESI/GSD) inclusion and demo projects for the PLC. The document also points to further *information on Heliotis gen4 devices and fieldbuses*.

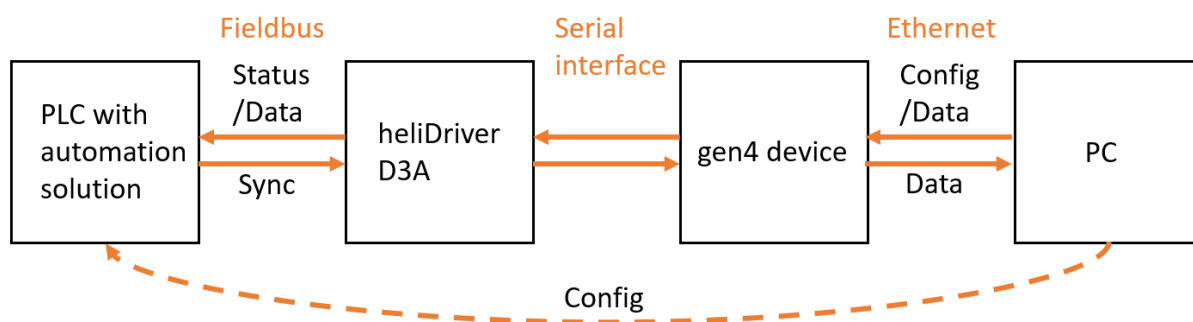


Fig. 10.1: System interconnection

Note: This document only covers gen4 device control via fieldbus but control and status features can also be used with physical I/Os or a combination of both

The following process data objects (PDOs) are available on the network:

Fieldbus names	Camera feature names	Direction
Bool0..7	FieldbusBool0..7	device in / PLC out
Bool8..15	FieldbusBool8..15	device out / PLC in
ApplDataInt0..15	FieldbusIntData0..15	device in / PLC out
ApplDataInt16..31	FieldbusIntData16..31	device out / PLC in
ApplDataFloat0..15	FieldbusFloatData0..15	device in / PLC out
ApplDataFloat16..31	FieldbusFloatData16..31	device out / PLC in

10.2 Example Use Cases

The following chapters describe some common use cases and how to implement them on both the device and the PLC controlling it.

Note: The link to the device description files that need to be imported to the PLC development environment can be found in the [Further Reading](#) section of this document

10.2.1 “Hello World”

It is recommended to begin with a very basic application to verify that the data exchange between the device and the PLC works in both directions. The following pseudo-code snippets configure a camera output via GenICam features and verify the value of the input PDO on the PLC.

Listing 10.1: Setting an output PDO via camera features

```
FieldbusBoolSelector = 8           #Or any value 8..15
FieldbusBoolSource[FieldbusBoolSelector] = "UserOutput0" #Or any of UserOutput0..
↪ UserOutput7
UserOutputSelector = "UserOutput0" #Must match with the configuration above
UserOutputValue[UserOutputSelector] = True
#True enables (high) the output, False disables (low) the output
```

Listing 10.2: Reading an input PDO on the PLC

```
input_value = Bool8               #According to FieldbusBool from above Listing
↪ (Bool8..Bool15)
```

To check the other direction, an output PDO value is set by the PLC and read via the FieldbusBoolStatus feature on the device.

Listing 10.3: Setting an output PDO on the PLC

```
Bool0 = 1                        #Or any bool PDO Bool0..Bool7
```

Listing 10.4: Reading an input PDO via camera features

```
FieldbusBoolSelector = 0         #Or any value according to the listing above (0..7)
input_value = FieldbusBoolStatus[FieldbusBoolSelector]
```

10.2.2 Camera Status

Heliotis gen4 devices offer a range of status flags (e.g. RecordingActive) that can be mapped to output PDOs to relay status information to the PLC.

Listing 10.5: Mapping the RecordingActive status flag to a PDO via camera features

```
FieldbusBoolSelector = 8           #Or any value 8..15
FieldbusBoolSource[FieldbusBoolSelector] = "RecordingActive" #Maps the
↪ "RecordingActive" information to the selected PDO (e.g. 8)
```

Listing 10.6: Reading an input PDO on the PLC

```
recording_active = Bool8           #According to the selected FieldbusBool from above
↪ listing (Bool8..Bool15)
```

The diagram below shows the different status flags and how they fit into the acquisition cycle.

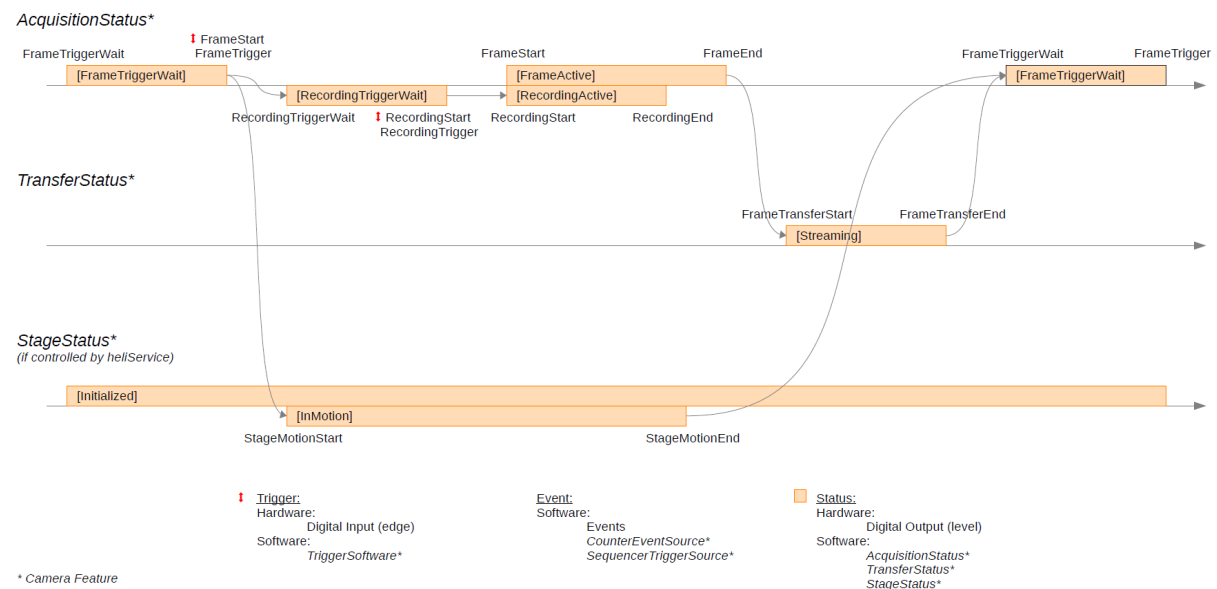


Fig. 10.2: Event and status diagram

Note: The current version of the Event and Status Diagram can be found in the Feature Description appendix (see [Further Reading](#)) and may differ depending on the camera firmware version.

10.2.3 Acquisition Control

To start a measurement on a Heliotis gen4 device, the TriggerSource feature can be used to map a PDO to a triggersource.

Listing 10.7: Mapping the FrameStart trigger to a PDO via camera features

```
TriggerSelector = "FrameStart"
TriggerSource[TriggerSelector] = "FieldbusBool0" #Maps the "FrameStart" trigger
↳to the FieldbusBool0 PDO (or any of FieldbusBool0..FieldbusBool7)
```

Listing 10.8: Setting an output PDO on the PLC to trigger the frame start

```
Bool0 = 1 #Select the PDO according to the camera configuration (Bool0..
↳Bool7)

Bool0 = 0 #By default, the camera is triggered on the "RisingEdge" of the
↳signal
#(See TriggerActivation in feature description for more information)
```

10.2.4 Camera Control, Light ON/OFF

The light can be controlled from the PLC via the LightControllerSource feature.

Listing 10.9: Mapping the light controller source to a PDO via camera features

```
LightControllerSelector = "LightController0"

LightControllerSource[LightControllerSelector] = "FieldbusBool0" #Maps the light
↳control to a PDO (any of FieldbusBool0..FieldbusBool7)
```

Listing 10.10: Setting an output PDO on the PLC to turn on the light

```
Bool0 = 1 #Enables the light

Bool0 = 0 #Disables the light
```

10.2.5 Camera Control, Sequencer Sets

Sequencer sets can be used on Heliotis gen4 devices to switch between multiple camera configuration states triggered by events (see *Event and Status Diagram*). The event source can be mapped onto an edge of a PDO signal.

Listing 10.11: Mapping the sequencer set trigger source to a PDO via camera features

```
SequencerSetSelector = 0
SequencerPathSelector = 0
SequencerTriggerSource[SequencerSetSelector][SequencerPathSelector] =
↪ "FieldbusBool0" #Maps the sequencer trigger to FieldbusBool0 (or any of
↪ FieldbusBool0..FieldbusBool7)
```

Listing 10.12: Setting an output PDO on the PLC to trigger the sequencer set execution

```
Bool0 = 1

Bool0 = 0 #By default the sequencer set transition is triggered by
↪ the "RisingEdge" of the signal. (see SequencerTriggerActivation)
```

10.2.6 Camera Control, Counters

The counter feature set provides the option to count the occurrences of specific events on the device. This can be helpful for testing and debugging of an application. It can for example be used to count transitions on a PDO.

Listing 10.13: Mapping the counter event source to a PDO via camera features and reading the counter value

```
CounterSelector = "Counter0" #Or Counter1
CounterEventSource[CounterSelector] = "FieldbusBool0" #Or any of FieldbusBool0..
↪ FieldbusBool7
CounterEventActivation[CounterSelector] = "RisingEdge" #Can also be "FallingEdge"
↪ or "AnyEdge"

cnt_value = CounterValue[CounterSelector]
```

Listing 10.14: Toggling an output PDO on the PLC to increment the counter

```
Bool0 = 1

Bool0 = 0
```

10.2.7 Vision Application, Parameterisation

The use of integer or float PDOs also enables the option to transfer data either to or from the vision application on the gen4 device. For example, the PLC could transfer a target value for a stepheight measurement as a float value. The vision application on the camera uses this value to compare it to the measurement result and set or reset a bool PDO depending on whether the measured value is higher or lower than the target value. The PLC can then use this return value to execute further actions.

Listing 10.15: Using camera features to realize a basic vision application

```
FieldbusBoolSelector = 8           #Or any value 8..15
FieldbusBoolSource[FieldbusBoolSelector] = "UserOutput0" #Or any value UserOutput0..
↪.UserOutput7
UserOutputSelector = "UserOutput0" #Must match with the configuration above
UserOutputValue[UserOutputSelector] = False #True enables (high) the output, False
↪disables (low) the output
FieldbusFloatDataSelector = 0 #Select the PDO which is used by the PLC to write
↪the step height information (any float PDO between 0..15)

acquisition loop:
    target_step_height = FieldbusFloatDataValue[FieldbusFloatDataSelector]
    measured_step_height = acquire_and_process()
    if (measured_step_height > target_step_height):
        UserOutputValue[UserOutputSelector] = True
    else:
        UserOutputValue[UserOutputSelector] = False
```

Listing 10.16: Transferring measurement parameters and receiving results on the PLC

```
acquisition loop:
    ApplDataFloat0 = TargetStepHeight           #Can be ApplDataFloat0..
↪ApplDataFloat15
    result = Bool8           #Can be Bool8..Bool15
    if (result == 0):
        execute action A
    else:
        execute action B
```

Note: This simplified example has no synchronisation for when the PLC should read the result. This can be achieved by using status features as shown in the [“Camera Status” example](#)

10.2.8 Vision Application, Result Transmission

Instead of parameterisation and evaluation on the gen4 device, measurement results can also be transmitted directly using integer or float output PDOs.

Listing 10.17: Using camera features to return measurement results

```
FieldbusFloatDataSelector = 16 #Select the PDO the results will be written to
↔ (any float PDO between 16..31)

acquisition loop:
    measured_step_height = acquire_and_process()
    FieldbusFloatDataValue[FieldbusFloatDataSelector] = measured_step_height
```

Listing 10.18: Transferring measurement parameters and receiving results on the PLC

```
acquisition loop:
    result = ApplDataFloat16 #Can be ApplDataFloat16..ApplDataFloat31
```

10.3 Further Reading

The following chapter contains links to other documents and hardware ordering information.

With the Heliotis SDK (C4Utility) available on <https://www.heliotis.com/support/updates/> (needs a login) further examples and documentation are installed. The installation directory is stored in the environment variable \$C4UTILITY_ROOT and is by default C:\Program Files\C4Utility

10.3.1 Heliotis

frtDesc: The documentation for the camera features can be found in \$C4UTILITY_ROOT\doc\frtDesc.pdf

ProgrammersGuide: Go to <https://www.heliotis.com/support/updates/> (needs a login) and download the zip archive labeled “heliInspect H8 Programmer’s Guide”

Examples: \$C4UTILITY_ROOT\examples offers examples for a variety of programming languages

Hardware ordering information: Independent of the chosen fieldbus, a heliDriver D3A is always required. Additionally only one of the available fieldbus modules needs to be added to the D3A. The choice of heliInspect (H8/H9) or heliCam (C4) devices has no influence on the fieldbus connectivity and is made independently.

device	type number TN	article number AN
heliDriver D3A	D3A.0	40 81 10
EtherCAT Module	D3MRTETH.0.0-ETHERCAT	40 81 90
PROFINET Module	D3MRTETH.0.0-PROFINET	40 81 95

10.3.2 Beckhoff

Documentation: The user manual for TwinCAT can be found on <https://infosys.beckhoff.com/english.php?content=../content>

Examples: An EtherCAT example project can be found in \$C4UTILITY_ROOT\fieldbus\examples\Beckhoff

Device description file: The EtherCAT Slave Information file (ESI) is located in \$C4UTILITY_ROOT\fieldbus\deviceDescription\Beckhoff\heliotisGen4Device_230926.xml

10.3.3 Siemens

Documentation: The TIA Portal user manual can be found on https://cache.industry.siemens.com/dl/files/542/40263542/att_829827/v1/GS_STEP7Bas105enUS.pdf

Examples: An example project for PROFINET is located in \$C4UTILITY_ROOT\fieldbus\examples\Siemens

Device description file: The General Station Description file (GSD) is located in \$C4UTILITY_ROOT\fieldbus\deviceDescription\Siemens\GSDML-V2.41-Heliotis AG-heliotis gen4 device-20231023.xml

10.4 Document History

Version	Date	Changes	Responsible
51.1.0.0	27-02-2024	Initial version	sb

EtherCAT® is a registered trademark and patented technology, licensed by Beckhoff Automation GmbH, Germany.